

Edge-Native Federated Learning: Synchronized On-Device Training and Real-Time Inference on Dual ARM Cortex-M33 Microcontrollers

Alhussein Ali, Nilüfer Tak
Karlsruhe Institute of Technology (KIT)
Chair of Embedded systems (CES)
{ufwyh, ujmqlf}@student.kit.edu

Abstract—Wearable human activity recognition (HAR) systems continuously sample intimate physiological and behavioral signals, making centralized cloud training of personalized models both privacy-invasive and energy-costly for battery-constrained devices. This paper presents the design, implementation, and empirical validation of an end-to-end, multi-client on-device Federated Learning (FL) testbed built entirely from commodity microcontroller hardware. Two STMicroelectronics B-U585I-IOT02A boards, each pairing an ARM Cortex-M33 core with an MXCHIP EMW3080B Wi-Fi module, run a hybrid inference-and-training pipeline in which a frozen, quantized convolutional backbone produces fixed embeddings that feed a small trainable linear classification head. This head is updated in place using closed-form stochastic gradient descent executed directly on the microcontroller, gated by human-in-the-loop gesture labeling through a browser-based dashboard. Model updates are exchanged with a Python aggregation server over persistent TCP sockets running on Azure RTOS ThreadX and NetX Duo, using a lockstep Federated Averaging (FedAvg) protocol. We document the complete system architecture, the closed-form training formulation, the wire protocol, and a set of embedded networking failure modes discovered and resolved during bring-up – including TCP stream fragmentation, flow-control stalls, and premature connection teardown. Empirical results across two synchronized FedAvg rounds show a monotonic decrease in the global parameter L_2 delta from 0.1154 to 0.0684, together with per-gesture cross-entropy loss trajectories confirming convergent on-device learning.

I. INTRODUCTION

A. The Need for Edge-Native Federated Learning

Human activity recognition from wearable inertial sensors is intrinsically privacy-sensitive: continuous accelerometer and gyroscope streams can reveal gait signatures, mobility patterns, and indirectly, health status. Transmitting raw inertial measurement unit (IMU) telemetry to a centralized cloud service for model training introduces three compounding costs. First, continuous radio transmission of high-frequency sensor data is a dominant contributor to wearable power budgets. Second, network round-trip latency is incompatible with responsive, real-time personalization. Third, aggregating raw personal sensor data at a central point creates regulatory exposure under privacy frameworks such as the GDPR and HIPAA. Federated Learning (FL) addresses these costs directly by inverting the data flow: rather than centralizing raw data, only

model parameter updates are exchanged, and training data never leaves the device [1], [2].

B. Challenges of On-Device Training on Resource-Constrained MCUs

Realizing FL on a microcontroller-class target departs sharply from FL's usual smartphone or edge-server setting. The STM32U585 used in this work exposes on the order of hundreds of kilobytes of SRAM and no hardware floating-point acceleration beyond a single-precision FPU. Full back-propagation through a convolutional feature extractor requires retaining every intermediate activation tensor for the backward pass, a memory cost that scales with network depth and rapidly exceeds what is available. Simultaneously, the microcontroller must service 26 Hz inertial sampling, real-time RTOS scheduling, I2C and SPI peripheral buses, and the network stack without dropping sensor windows. Recent work on memory-efficient on-device learning has shown that freezing a pretrained feature extractor and adapting only a small parameter-efficient head preserves most of the adaptation capacity of full fine-tuning while eliminating the need to store backbone activations for backpropagation [4]; a philosophy this work adopts directly. Lightweight embedded inference runtimes such as TensorFlow Lite Micro have similarly demonstrated that quantized neural inference is practical within these memory envelopes [5], which motivates our own choice of a compact, quantized convolutional backbone.

C. Key Contributions

This work makes three contributions: (1) a split-architecture edge model combining a static, quantized convolutional feature backbone with an on-device trainable linear head updated via closed-form softmax cross-entropy SGD; (2) a real-time, multi-client embedded FL runtime implemented on ARM Cortex-M33 silicon using Azure RTOS ThreadX and NetX Duo, including a fixed-length reliable receive primitive that corrects for TCP stream fragmentation over an SPI-bridged Wi-Fi link; and (3) a complete hardware-software co-design encompassing interactive, human-in-the-loop gesture labeling via per-device web dashboards, concurrent multi-client TCP synchronization, and server-side FedAvg aggregation, validated empirically on physical hardware.

TABLE I
SYSTEM HARDWARE AND RESOURCE ALLOCATION

Component	Specification
MCU	STM32U585AII6, Cortex-M33 @ 160 MHz
Security/FPU	TrustZone, single-precision FPU
Flash / SRAM	2 MB / 786 KB
IMU	ISM330DHCX, 6-axis, I2C @ 26 Hz
Wi-Fi module	MXCHIP EMW3080B (802.11b/g/n, SPI2)
RTOS / stack	Azure RTOS ThreadX / NetX Duo
FL payload	56 params. (224 B), IEEE-754 float32

II. SYSTEM ARCHITECTURE AND HARDWARE-SOFTWARE CO-DESIGN

A. Hardware Infrastructure

The testbed comprises two STMicroelectronics B-U585I-IOT02A Discovery Kits. Each board is built around an STM32U585AII6 microcontroller (ARM Cortex-M33 core, TrustZone, single-precision FPU, 2 MB flash, 786 KB SRAM) and integrates an ST ISM330DHCX six-axis inertial sensor (three-axis accelerometer and gyroscope) sampled over I2C at 26 Hz. Wireless connectivity is provided by an MXCHIP EMW3080B 802.11b/g/n module, bridged to the host MCU over a dedicated SPI2 bus with auxiliary reset, flow-control, and notify GPIO lines. A Python-based aggregation server runs on a Linux host that simultaneously acts as the local Wi-Fi access point, giving both boards a stable, low-latency link to the FL server on the same subnet. Table I summarizes the resource envelope.

B. RTOS Concurrency Architecture

Firmware concurrency is organized as a set of preemptive ThreadX threads. A high-priority sensor thread performs the 26 Hz I2C acquisition and windowing pipeline and invokes the frozen backbone’s quantized inference kernel once per completed window. A command thread parses asynchronous UART input from the host dashboard for gesture labeling, session control, and weight-inspection commands. A dedicated NetX Duo thread manages the FL client TCP socket, handshake, and per-round weight exchange. Two synchronization primitives coordinate these threads: a mutex guards the trainable head’s weight matrix against concurrent access from the inference/training path and the network thread’s import/export calls, and a counting semaphore gates the network thread until the operator signals, via the dashboard, that local training for the current round is complete. Figure 1 depicts the resulting dual-board pipeline.

III. MODEL TOPOLOGY AND ON-DEVICE TRAINING FORMULATION

A. Hybrid Frozen Backbone and Trainable Linear Head

Each board runs a quantized temporal convolutional backbone, executed through the ST Edge AI inference runtime, which maps a 24-sample window of three-axis accelerometer dynamics to a 12-dimensional latent embedding $z \in \mathbb{R}^{12}$. This

embedding is concatenated with a single derived gyroscope-magnitude feature, $x_{12} = \sqrt{\omega_x^2 + \omega_y^2 + \omega_z^2}$, yielding a 13-dimensional feature vector $x \in \mathbb{R}^{13}$ consumed by a trainable linear classification head over four activity classes,

$$\mathcal{C} = \{\text{Jogging (0), Stationary (1), Stairs (2), Walking (3)}\}.$$

The head computes

$$y = Wx + b, \quad W \in \mathbb{R}^{4 \times 13}, \quad b \in \mathbb{R}^4, \quad (1)$$

$$\hat{p}_k = \frac{e^{y_k}}{\sum_{j=0}^3 e^{y_j}}. \quad (2)$$

Restricting on-device backpropagation to this linear head – rather than fine-tuning the convolutional backbone – follows directly from the memory argument in [4]: since no backbone activations need to be retained for a backward pass, the trainable parameter set collapses to $4 \times 13 + 4 = 56$ single-precision floats, a 224-byte payload that is both cheap to hold in SRAM and cheap to transmit per FL round.

B. On-Device Backpropagation and Loss Formulation

Training uses softmax cross-entropy loss against a one-hot target t supplied by the operator’s live gesture label,

$$\mathcal{L}_{CE} = - \sum_{k=0}^3 t_k \ln \hat{p}_k. \quad (3)$$

Because the head is linear and the loss is softmax cross-entropy, the gradient with respect to the pre-softmax logits reduces to the closed form

$$\frac{\partial \mathcal{L}}{\partial y_k} = \hat{p}_k - t_k, \quad (4)$$

which yields the parameter gradients

$$\frac{\partial \mathcal{L}}{\partial W_{k,i}} = (\hat{p}_k - t_k) x_i, \quad \frac{\partial \mathcal{L}}{\partial b_k} = \hat{p}_k - t_k. \quad (5)$$

No autodifferentiation graph or intermediate-activation storage is required: each training step is a fixed sequence of dot products and outer-product updates. The SGD update with learning rate η ,

$$W \leftarrow W - \eta \nabla_W \mathcal{L}, \quad b \leftarrow b - \eta \nabla_b \mathcal{L}, \quad (6)$$

executes under a dedicated mutex to serialize access against the concurrent inference path, and completes well within a single sensor window period at the platform’s clock rate.

IV. FEDERATED LEARNING PROTOCOL AND NETWORKING CHALLENGES

A. Wire Framing and Protocol Design

The aggregation server listens on a persistent TCP port and, on accepting each of K client connections, sends a 4-byte little-endian handshake encoding the total round count R – chosen to match the Cortex-M33’s native byte order so that received payloads require no endianness conversion. Each of the R subsequent rounds follows a lockstep exchange: the server broadcasts the current global parameter

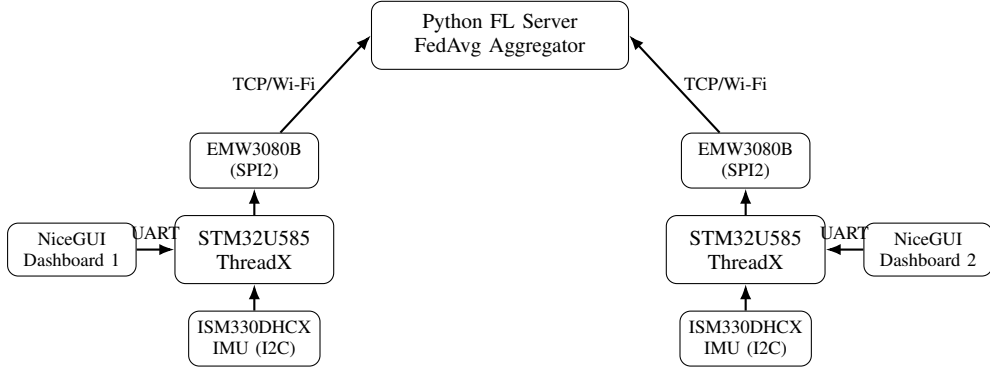


Fig. 1. Dual-MCU to Python-server FL pipeline: each board independently samples its IMU, runs frozen-backbone inference, accepts human-in-the-loop labels over UART, and exchanges trainable-head weights with the aggregation server over a persistent TCP/Wi-Fi link.

vector $\Theta_G^r \in \mathbb{R}^{56}$; each client imports it directly into its trainable head; local corrective SGD steps run as the operator labels physical gestures through the dashboard; the operator’s “round complete” action releases a counting semaphore that unblocks the network thread; the client exports its locally-adapted parameters Θ_i^r and transmits them back; and once all K clients have submitted, the server computes

$$\Theta_G^{r+1} = \frac{1}{K} \sum_{i=1}^K \Theta_i^r, \quad (7)$$

checkpoints the result, and proceeds to round $r+1$. This lockstep, barrier-synchronized structure is a direct embedded realization of the original FedAvg algorithm [1]; the broader design space of client heterogeneity, partial participation, and communication compression is surveyed extensively in [3], though the present testbed targets the simpler $K=2$, fully-synchronous regime as a first hardware validation step.

B. Embedded Network Stack Engineering and Failure-Mode Resolution

Several failure modes specific to running a byte-stream transport over an SPI-bridged Wi-Fi module surfaced during bring-up and were diagnosed directly from firmware evidence rather than assumed.

TCP fragmentation and stream boundaries. An initial implementation assumed that a single call to the NetX Duo receive primitive would return an entire 224-byte payload. TCP provides no message-boundary guarantee, and a payload can arrive split across multiple underlying packets depending on link-layer segmentation. Algorithm 1 shows `fl_tcp_receive_exact`, which accumulates bytes across repeated receive calls until exactly the requested length has been extracted, eliminating payload truncation regardless of how the transport segments the stream.

Flow-control stalls under a small receive window. The client socket’s receive window was initially sized from the same small constant used for a fixed per-packet payload buffer, well under a single Ethernet-scale frame. Enlarging the socket window to 1536 bytes removed a class of stalls consistent with

Algorithm 1 `fl_tcp_receive_exact`: reliable fixed-length receive

Require: socket S , destination buffer D , byte count N , wait option τ

Ensure: exactly N bytes copied into D , or a non-success status

```

1:  $r \leftarrow 0$ 
2: while  $r < N$  do
3:    $(status, P) \leftarrow \text{NX\_TCP\_SOCKET\_RECEIVE}(S, \tau)$ 
4:   if  $status \neq \text{SUCCESS}$  then return  $status$ 
5:   end if
6:    $(status, c) \leftarrow \text{NX\_PACKET\_DATA\_EXTRACT\_OFFSET}(P, 0, D+r, N-r)$ 
7:    $\text{NX\_PACKET\_RELEASE}(P)$ 
8:   if  $status \neq \text{SUCCESS}$  then return  $status$ 
9:   end if
10:  if  $c = 0$  then return NOTSUCCESSFUL ▷ avoid
    infinite loop
11:  end if
12:   $r \leftarrow r + c$ 
13: end while
14: return SUCCESS

```

TCP flow-control back-pressure against an undersized receive buffer on the embedded endpoint.

Nagle’s algorithm and teardown timing. On the aggregation server, enabling `TCP_NODELAY` avoids Nagle-induced coalescing delays on the small, latency-sensitive 224-byte round messages, and configuring `SO_LINGER` on each accepted socket ensures buffered data – including the final aggregated model sent after the last round – is flushed before the server-side socket is torn down, rather than risking an abrupt reset.

Physical bus isolation. Running two boards concurrently on a Linux host, USB CDC-ACM enumeration order is not guaranteed stable across reconnects. Binding to each board’s persistent `/dev/serial/by-id/` path, rather than the transient `/dev/ttyACM*` device node, removes a source of intermittent dashboard-to-board mismatches during dual-board sessions.

TABLE II
MULTI-ROUND FEDERATED AVERAGING CONVERGENCE DATA

Round	$\ \Delta_{c0}\ _2$	$\ \Delta_{c1}\ _2$	Global $L_2 \Delta$
1	0.1608	0.0905	0.115404
2	0.1864	0.1238	0.068371

V. EMPIRICAL RESULTS AND DISCUSSION

A. Multi-Client Convergence Metrics

Table II reports the first two synchronized FedAvg rounds observed on physical hardware, including each client’s local update norm and the resulting global parameter delta. The global L_2 delta decreased monotonically from 0.1154 in round 1 to 0.0684 in round 2, the expected convergence signature of averaging two independently-adapted local models toward a shared consensus point under repeated aggregation.

B. Local Training Dynamics

Individual gesture labels produced clear, monotonic cross-entropy loss reduction during on-device SGD. For the Jogging class, loss fell from an initial value of 1.3863 – notably close to $\ln 4 \approx 1.386$, the loss of an uninformative uniform four-class prior – to a converged value of 0.0354 after repeated corrective steps on the same gesture. The Stationary class converged to a final loss of 0.6641, a higher residual consistent with greater feature overlap between stationary and low-dynamic-range gesture windows at the 12-dimensional embedding stage.

C. Weight Synchronization Verification

Post-aggregation parameter parity between boards was verified by round-tripping each board’s live head weights over its UART GET_WEIGHTS command and comparing the resulting 224-byte payloads against the server-side checkpoint, and independently by comparing the softmax output vectors both boards produced for matched inference windows after applying the same aggregated global model. Both checks confirmed byte-identical (weights) and numerically consistent (softmax) synchronization across devices.

D. Discussion

These results demonstrate that a complete FL loop – local adaptation, network transport, aggregation, and redistribution – is achievable on commodity Cortex-M33 silicon without a host-resident training component, provided the trainable surface is deliberately restricted to a memory-cheap linear head and the transport layer is hardened against the byte-stream assumptions that a naive single-call receive implicitly makes.

VI. CONCLUSION

This paper presented a complete hardware deployment of edge-native Federated Learning across two Cortex-M33 micro-controllers, combining a frozen quantized convolutional backbone with an on-device trainable linear head, a ThreadX/NetX Duo FL client, and a Python FedAvg aggregation server, with no cloud dependency at any stage of training. Empirical results

confirmed correct multi-round FedAvg execution, monotonic global parameter convergence, and convergent per-gesture on-device loss trajectories, validated on physical hardware rather than in simulation. Future work includes integrating differential-privacy noise into the exported update, quantizing the transmitted gradient/weight payload to reduce per-round bandwidth, and extending the synchronous $K=2$ protocol to asynchronous client selection to scale toward $N > 10$ devices.

REFERENCES

- [1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proc. 20th Int. Conf. Artif. Intell. Statist. (AISTATS)*, ser. PMLR, vol. 54, Fort Lauderdale, FL, USA, Apr. 2017, pp. 1273–1282.
- [2] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, “Federated learning: Strategies for improving communication efficiency,” in *NeurIPS Workshop on Private Multi-Party Machine Learning*, Barcelona, Spain, Dec. 2016.
- [3] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings *et al.*, “Advances and open problems in federated learning,” *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [4] H. Cai, C. Gan, L. Zhu, and S. Han, “TinyTL: Reduce memory, not parameters for efficient on-device learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [5] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, T. Wang, P. Warden, and R. Rhodes, “TensorFlow Lite Micro: Embedded machine learning for TinyML systems,” in *Proc. Machine Learning and Systems (MLSys)*, vol. 3, 2021, pp. 800–811.
- [6] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, “A public domain dataset for human activity recognition using smartphones,” in *21st European Symp. Artificial Neural Networks, Computational Intelligence and Machine Learning (ESANN)*, Bruges, Belgium, Apr. 2013.